

Web программирование

Курс для бакалавриата

Чернышев Вячеслав

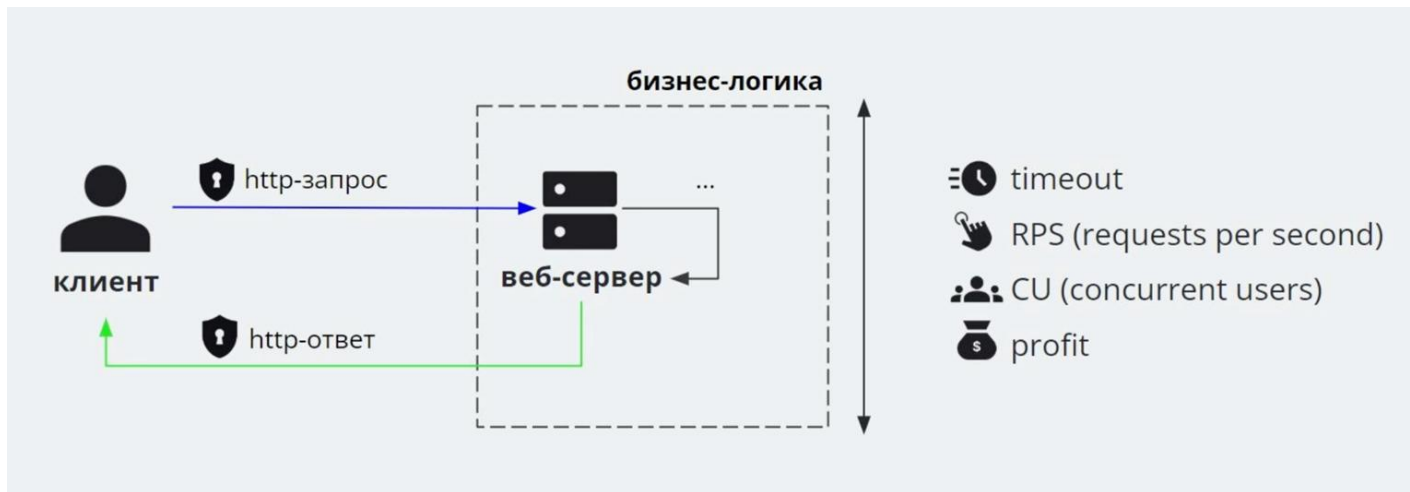
Фоновые задачи, очереди

Содержание лекции

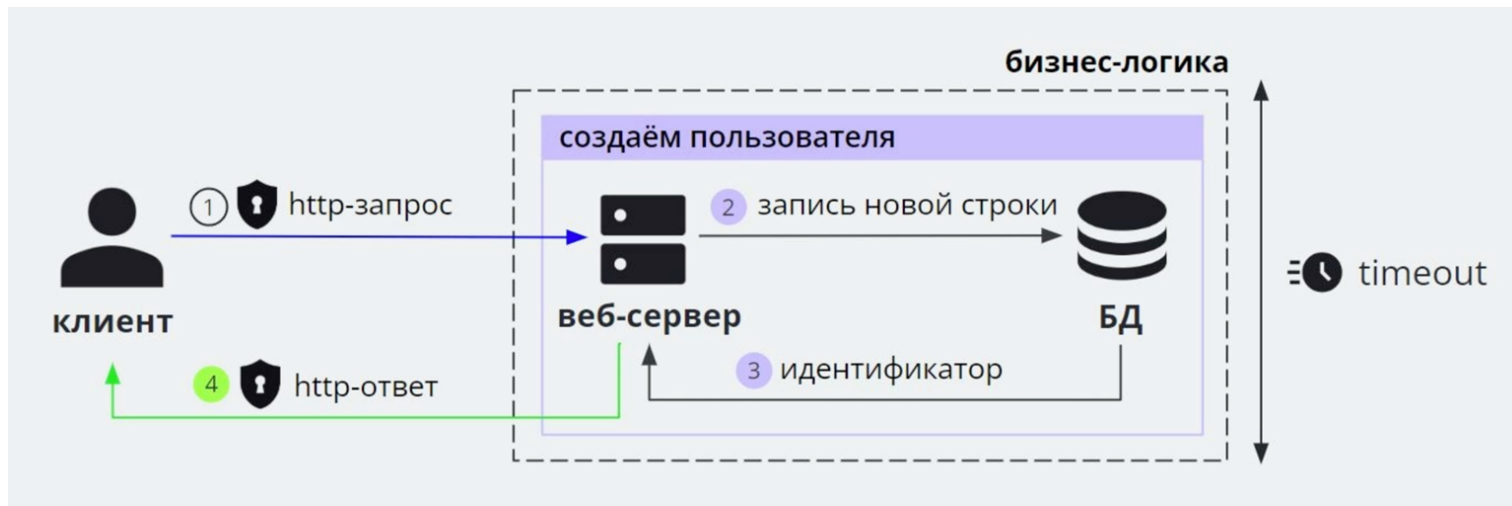
1. Фоновые задачи
2. FastAPI Background Tasks
3. Брокеры
4. Redis RQ
5. Celery

Фоновые задачи

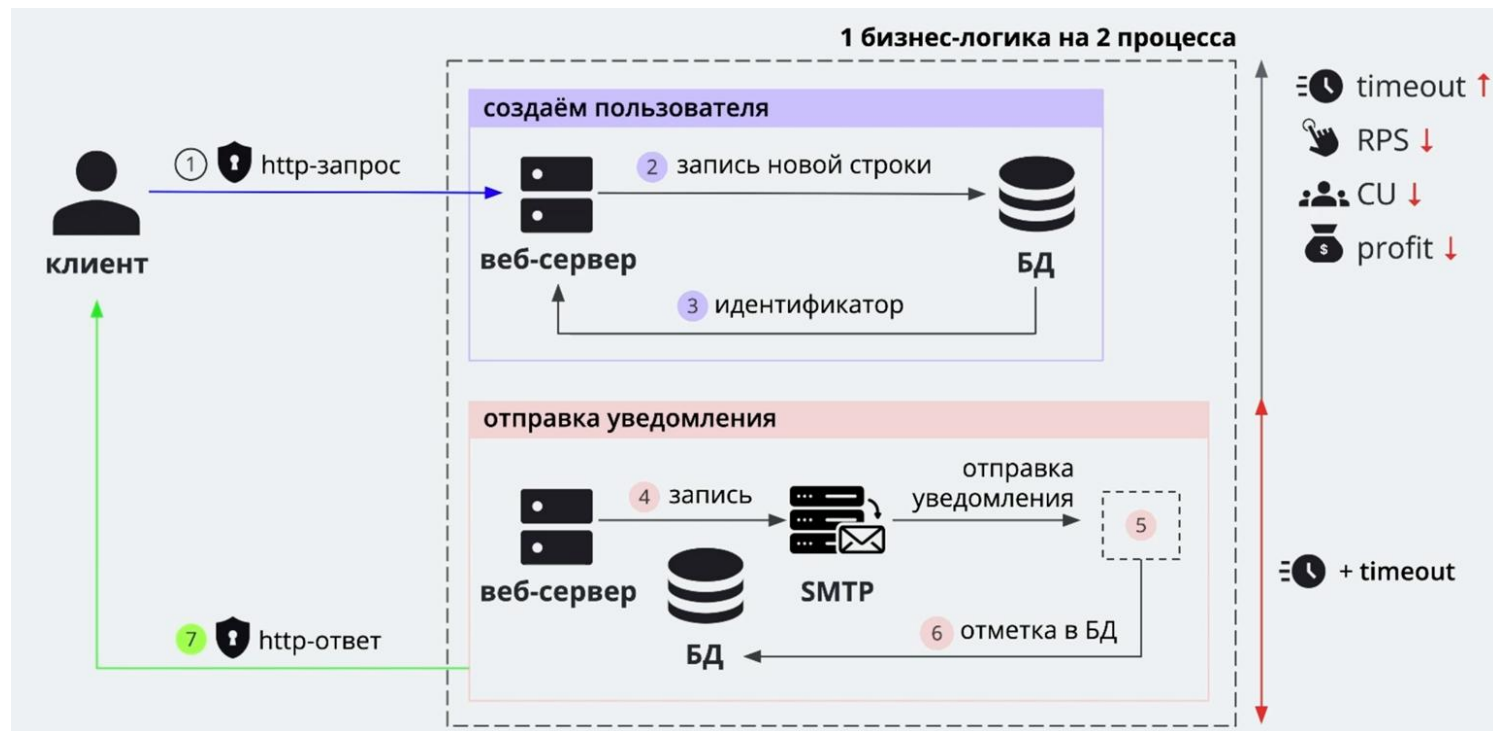
Метрики HTTP запроса



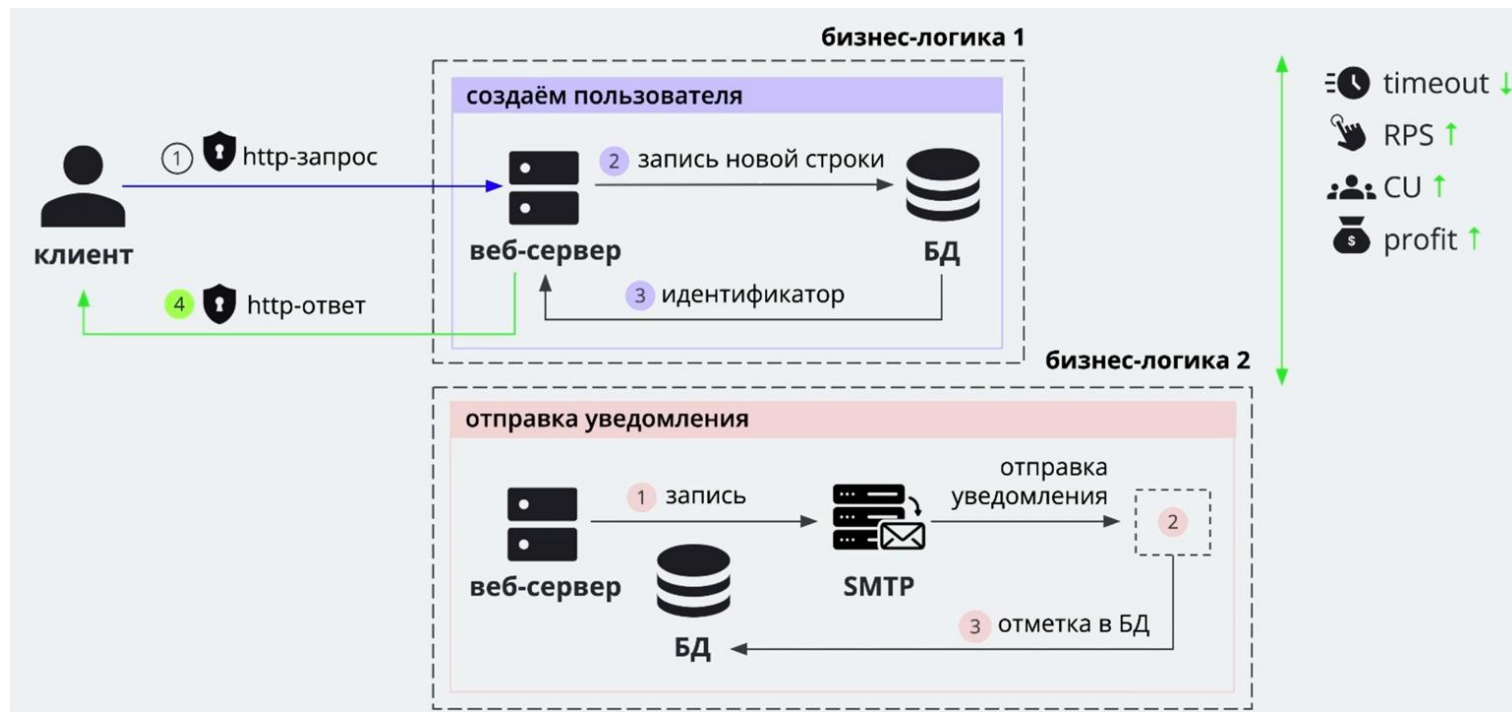
Типовой запрос - регистрация пользователя



Добавляем отправку по email



Долгий запрос - решение



Асинхронная архитектура

Цель - сохранить быстродействие системы при возрастающей нагрузке

Для внедрения нужно:

- Выделить медленные участки
- Вынести их обработку в фоновые задачи
- Обеспечить надежное выполнение фоновых задач

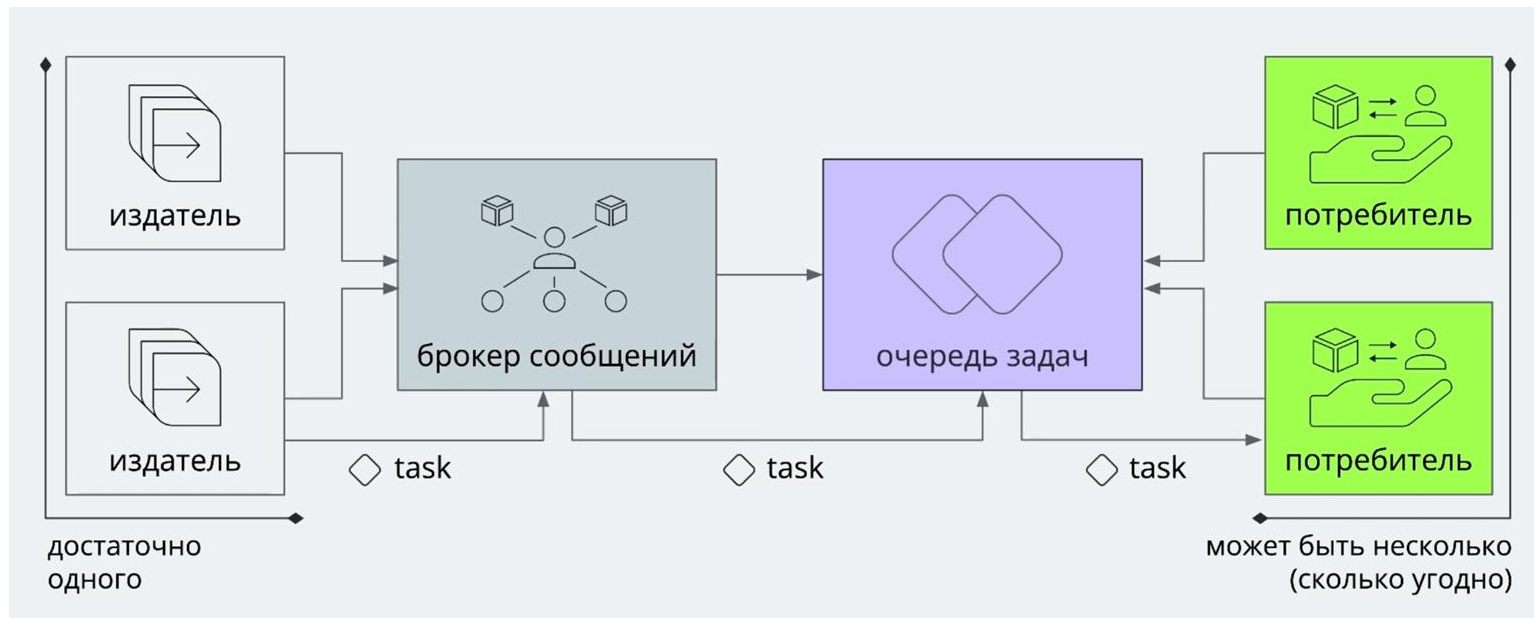
Выбираем асинхронную архитектуру для

- Уменьшения времени отклика на запрос
- Параллелизация медленной логики
- Реализация на альтернативном стеке

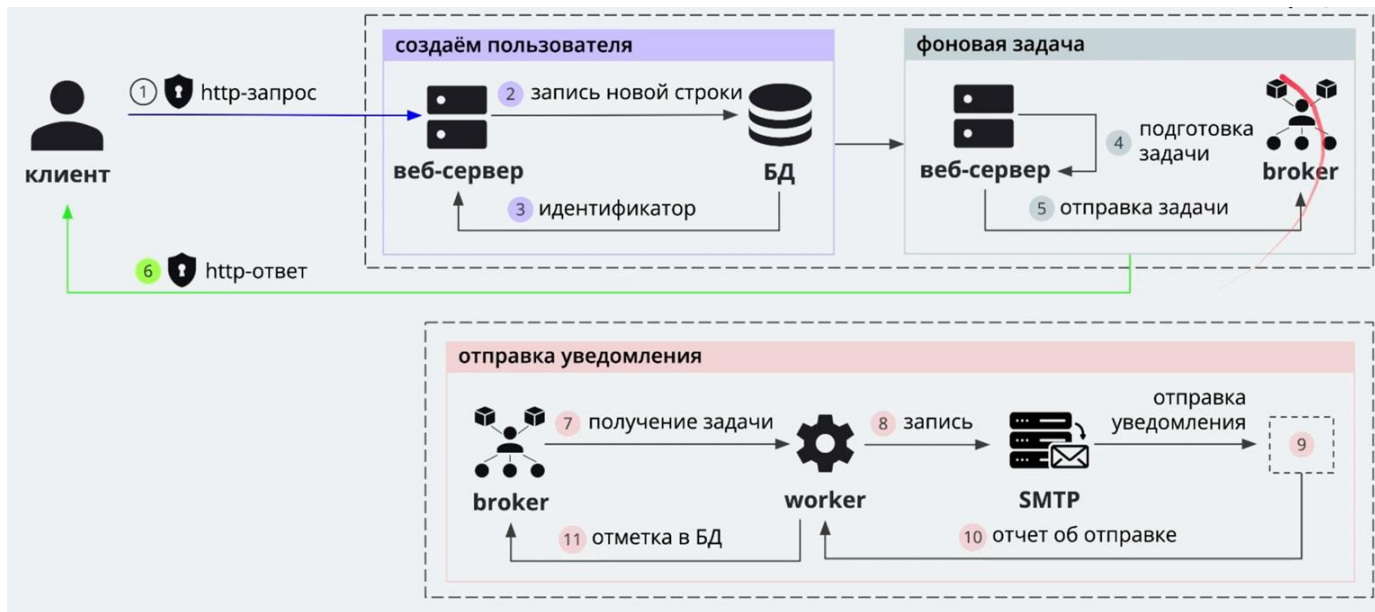
Терминология асинхронных очередей

- **Брокер** - приложение предоставляющее очередь
- **Издатель** (producer) - формирует фоновые задачи
- **Потребитель** (consumer) - получает задачи из очереди

Архитектурная схема



Регистрация с отправкой уведомлений



Когда использовать асинхронную архитектуру

- Процессы можно поделить на быструю и медленную части
- Ответ клиенту содержит результаты быстрой логики
- Клиент может сопоставить результаты ответа и фоновой задачи

Когда не использовать асинхронную архитектуру

- Все процессы обработки работают одинаково быстро
- Результаты медленной логики присутствуют в ответе
- Результаты быстрой и медленной логики несопоставимы

FastAPI Background Tasks

Особенности

- Нет брокера, есть отложенный запуск
- Обработка запускается сразу после ответа
- Выполнение фоновых задач ненадежно (данные задач в памяти)
- Процесс выполнения не параллелизуется - один обработчик на одну задачу

Преимущества

- Простота
- Нет изменений в инфраструктуру
- Запускаются в том же процессе

Преимущества FastAPI ВТ = их недостатки

Простота -> нет никаких плюшек

Нет изменений в инфраструктуру -> нет параллелизации

Запускаются в том же процессе -> неэффективны на CPU-bound задачах

Преимущества

- Простота
- Нет изменений в инфраструктуру
- Запускаются в том же процессе

Пример

Не ок

```
@app.post("/add-user-legacy")
async def add_user_legacy(user: User):
    user_id = await add_user(user)
    send_notification(user.email)
    return { "id": str(user_id) }
```

Ок

```
@app.post("/add-user-bg")
async def add_user_bg(user: User, tasks: BackgroundTasks):
    user_id = await add_user(user)
    tasks.add_task(send_notification, user.email)
    return { "id": str(user_id) }
```

Брокеры

Профессиональные инструменты для очередей



RQ (Redis)

Легкий брокер
поверх Redis



RabbitMQ

Брокер на все
случаи жизни



Kafka

Для высоких
нагрузок

Сравнение

Название	Сложность	Описание	Подходит если
FastAPI BT		Фоновые задачи из коробки с нулевой сложностью	Быстро разгрузить роут от задач, связанных с IO
RQ		Очереди на базе Redis Быстрое внедрение Скромный функционал	Быстро вывести в отдельный процесс задачи, связанные с CPU
RabbitMQ		Очереди на все случаи Богатый функционал Простое внедрение Свой UI	Реализовать асинхронную архитектуру с распределенной обработкой данных между несколькими сервисами
Kafka		Высокопроизводительное решение с высоким уровнем надежности	Реализовать высокопроизводительную систему с распределенными вычислениями

Структура данных очередь

- FIFO - first in, first out

Примеры из Redis:

LPUSH tasks "send_email:user123"

RPOP tasks

- LPUSH queue value — добавить элемент в начало.
- RPUSH — в конец.
- LPOP / RPOP — извлечь элемент.

BRPOP — блокирующее извлечение (ждёт, пока появится элемент)

Redis RQ

Worker - тот, кто слушает очередь и выполняет задачи

```
@app.post("/send-email")
async def send_email(email: EmailRequest):
    try:
        from tasks import send_email_task

        job = queue.enqueue(
            send_email_task,
            email.to,
            email.subject,
            email.body
        )

        return {
            "job_id": job.id,
            "message": f"Email задача добавлена в очередь",
            "status": "queued",
            "to": email.to
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error: {str(e)}")
```

```
# Создаем очередь
queue = Queue('emails', connection=redis_conn)

print("=" * 60)
print("EMAIL WORKER ЗАПУЩЕН")
print("=" * 60)
print(f"Слушаю очередь: {queue.name}")
print(f"Redis: localhost:6379")
print("Нажмите Ctrl+C для остановки\n")

# Создаем и запускаем worker
worker = Worker([queue], connection=redis_conn)
worker.work()
```

Фреймворк для работы с фоновыми задачами

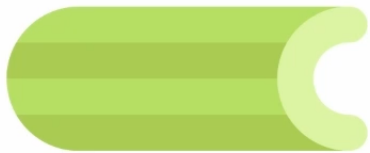
Нужен если:

- Нужен быстрый результат без тонкостей взаимодействия
- Требуется типовое решение для эффективной поддержки ряда проектов

Не нужен:

- Для простых задач
- Используются уникальные особенности брокера
- Если используется kafka

Известные Python-фреймворки



Celery

Наиболее
распространенное
и развитое
решение



Dramatiq

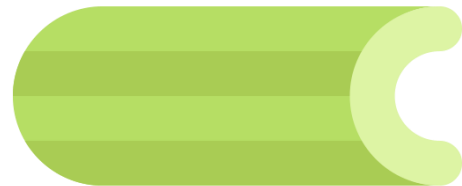
Простая и
современная
альтернатива
Celery



FastStream

Новый,
предоставляет
интерфейс ко всем
брокерам

Celery



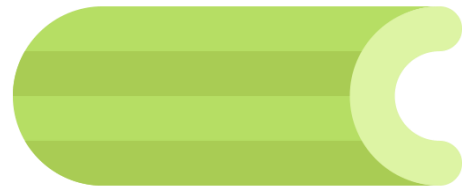
Преимущества:

- Развитый и проверенный временем
- Отличная поддержка
- Огромный набор инструментов
- Широкая интеграция

Минусы:

- Не работает с Kafka
- Сложный в настройках

Плюшки Celery



- Синхронный режим запуска
- Сериализация (форматы данных, сжатие, шифрование)
- Хранение результатов в БД
- Запуск по расписанию (crontab)
- Совместная работа задач (link, group)

Расписание

celery -A tasks beat --loglevel=info

```
from celery.schedules import crontab

# Конфигурация для периодических задач Celery Beat
beat_schedule = {
    # Отправка отчета каждую минуту
    'minute-report': {
        'task': 'tasks.send_email_task',
        'schedule': crontab(minute=0),
        'args': ('admin@example.com', 'Минутный отчет', 'Отчет за последнюю минуту')
    },
    # Отправка отчета каждый день в 9:00
    'daily-report': {
        'task': 'tasks.send_email_task',
        'schedule': crontab(hour=9, minute=0),
        'args': ('admin@example.com', 'Ежедневный отчет', 'Отчет за вчерашний день')
    },
    # Отправка напоминания каждый час
    'hourly-reminder': {
        'task': 'tasks.send_email_task',
        'schedule': crontab(minute=0), # Каждый час в начале часа
        'args': ('team@example.com', 'Почасовое напоминание', 'Проверьте систему')
    },
    # Еженедельный отчет по понедельникам в 10:00
    'weekly-report': {
        'task': 'tasks.send_email_task',
        'schedule': crontab(hour=10, minute=0, day_of_week=1),
        'args': ('boss@example.com', 'Еженедельный отчет', 'Отчет за прошлую неделю')
    },
}
```

Retry c backoff

```
@app.task(name='tasks.with_retry', bind=True, max_retries=3)
def with_retry(self, x: int):
    try:
        result = 100 / x
        return result
    except ZeroDivisionError as exc:
        retry_count = self.request.retries
        countdown = 10 * (2 ** retry_count)

        raise self.retry(
            exc=exc,
            countdown=countdown,
            max_retries=3
        )
```


Retry c backoff

```
@app.task(name='tasks.with_retry', bind=True, max_retries=3)
def with_retry(self, x: int):
    try:
        result = 100 / x
        return result
    except ZeroDivisionError as exc:
        retry_count = self.request.retries
        countdown = 10 * (2 ** retry_count)

        raise self.retry(
            exc=exc,
            countdown=countdown,
            max_retries=3
        )
```

Idempotency-key

- При создании транзакции или письма храним `idempotency_key` в БД.
- Если задача с таким ключом уже выполнялась -> пропускаем.

Graceful shutdown

- Прервать прием новых задач
- Закончить выполнять текущие задачи

```
# Регистрируем обработчики сигналов для graceful shutdown
signal.signal(signal.SIGINT, signal_handler) # Ctrl+C
signal.signal(signal.SIGTERM, signal_handler) # kill/systemd
```

```
def signal_handler(signum, frame):
    """
    Обработчик сигналов для graceful shutdown
    """
    signal_name = signal.Signals(signum).name

    print(f"\n{'='*60}")
    print(f"🔔 Получен сигнал {signal_name} ({signum})")
    print("👋 Graceful shutdown: завершаю текущие задачи...")
    print("Для принудительной остановки нажмите Ctrl+C еще раз")
    print('='*60)

    # Worker завершит текущие задачи и остановится
    sys.exit(0)
```

Запускаем

- 1. Redis
- 2. Приложение: `uvicorn main:app --reload`
- 3. Воркера: `python consumer.py`
- 4. Мониторинг: `celery -A tasks flower`

Show

15

 tasks

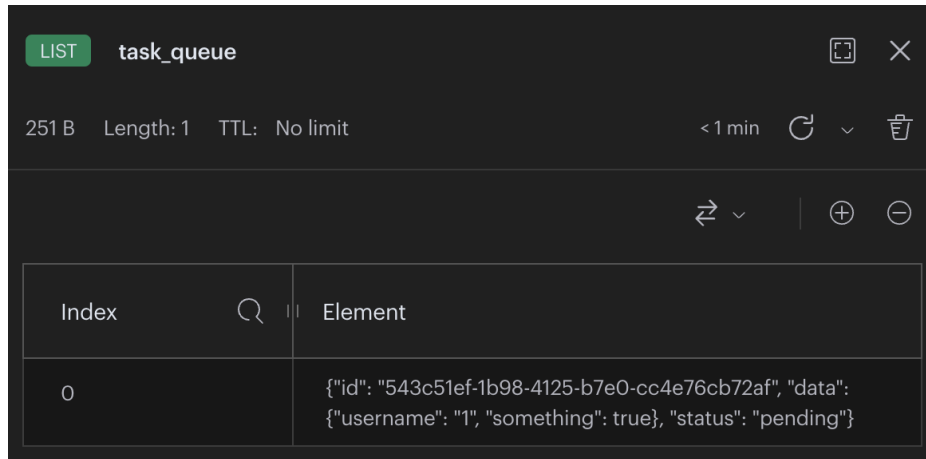
Search:

Name	UUID	State	args	kwargs	Result	Received
write_notification	ae705201-d7ed-45cc-a0d1-bd57ebe7d2d1	STARTED	('a@a.ru',)	{'message': 'some notification'}		2024-09-30 14:38:49.054
write_notification	b7dc3597-3b6a-4dbd-ac56-f20c2a686b6d	SUCCESS	('a@a.ru',)	{'message': 'some notification'}	'Notification sent to a@a.ru'	2024-09-30 14:38:48.826
write_notification	e8da087b-f4c5-4fb0-bd10-16084b85ef5b	SUCCESS	('a@a.ru',)	{'message': 'some notification'}	'Notification sent to a@a.ru'	2024-09-30 14:38:48.581

Напишем собственную реализацию функции добавления

```
# Функция для добавления задачи в очередь
def enqueue_task(task_data: Dict[str, Any]):
    task_id = str(uuid.uuid4())
    task = {
        "id": task_id,
        "data": task_data,
        "status": "pending"
    }

    redis_client.rpush(Queue_NAME, json.dumps(task))
    redis_client.set(f"task:{task_id}", json.dumps(task))
    return task_id
```



LIST task_queue

251 B Length: 1 TTL: No limit < 1 min

Index	Element
0	<pre>{ "id": "543c51ef-1b98-4125-b7e0-cc4e76cb72af", "data": { "username": "1", "something": true }, "status": "pending" }</pre>

Напишем собственную реализацию воркера

Функция воркера

```
def worker():
```

```
    print("Worker started")
```

```
    while True:
```

```
        # Получаем задачу из очереди
```

```
        _, task_json = redis_client.blpop(Queue_NAME)
```

```
        task = json.loads(task_json)
```

```
        # Обновляем статус задачи
```

```
        task["status"] = "processing"
```

```
        task["started_at"] = datetime.now().isoformat()
```

```
        redis_client.set(f"task:{task['id']}", json.dumps(task))
```

```
        # Выполняем задачу
```

```
        process_task(task["data"])
```

Функция для выполнения задачи

```
def process_task(task_data):
```

```
    # Здесь выполняется задача
```

```
    time.sleep(5)
```

```
    print(f"Task processed: {task_data}")
```

Coding samples

<https://github.com/itmo-webdev/webdev-2025>



Вопросы?